

Cache-Sharing as Failover: A Mesh Protocol for Token-Efficient LLM Clusters

PrismLib Micro & the CHORUS Mesh — Technical Whitepaper

Amin Parva

InsightIts · Version 1.0 · 2026

Apache 2.0 ·

om/insightitsGit/
.
htits.info@gmail.

Abstract

Production retrieval-augmented-generation (RAG) systems run as clusters of identical service replicas. Because each replica answers requests independently, the cluster performs the same expensive work — embedding, retrieval, and LLM inference — many times over for semantically identical queries. We present **PrismLib Micro**, a cluster layer that eliminates this redundancy by sharing LLM answers across nodes in real time over a lightweight machine-to-machine protocol we call **CHORUS**, and that reuses the same broadcast channel to provide leaderless hot-standby failover.

We report results from a three-node cluster deployed on **Azure Container Apps** across two separate virtual networks. On a workload where warm nodes receive queries already answered elsewhere in the cluster, those nodes serve responses for **zero additional tokens**; the active node reduces its own context cost by **58–64%** through similarity-based context compression. CHORUS frames cross between virtual networks in **~22 ms**, cluster health alerts propagate in **under 700 ms** without a metrics-scrape stack, and a silenced active node is replaced by a warm standby in **~4 seconds** (detection) plus **~100 ms** (promotion), with no orchestrator and no human intervention.

We are explicit throughout about the scope of each measurement. The cluster mesh ran on real cloud infrastructure (Azure Container Apps, westus2) with GREEN+BLUE in one VNet and ORANGE in a second; the two underlying single-node layers — the semantic cache and the streaming database driver — were separately load-tested on Azure under concurrent traffic (§4.6). Both VNets are in the **same region**, so cross-*region* latency and partition behavior remain untested. The benchmark code is open source and reproducible. Our aim is to document a design and its measured behavior honestly, not to claim a controlled scientific study.

1. Problem

A RAG service is typically deployed as N replicas behind a load balancer. This is good for availability and throughput, but it has a cost structure that scales badly with redundancy:

Duplicated inference. Two users asking "what is your refund policy?" and "how do refunds work?" are routed to different replicas. Each replica embeds, retrieves, and calls the LLM independently. The cluster pays for the same answer N times.

Oversized context. Each call typically stuffs the full retrieved document set into the prompt. Most of those tokens are irrelevant to the specific question and inflate the bill on every single call.

Heavy operational dependencies. The conventional fixes — Redis or GPTCache for caching, Prometheus/Datadog for health, Kubernetes or Raft for failover — each add an external system to run, secure, and pay for.

The cost of (1) and (2) is multiplicative: redundant calls, each larger than it needs to be. The cost of (3) is organizational: three more moving parts before a small team can run a cluster safely.

2. Design

PrismLib Micro addresses all three with a single shared channel. The core idea is that **the same broadcast used to share cache entries is also used to detect node failure** — one protocol, two problems solved.

2.1 The CHORUS mesh

Every node maintains connections to its peers and exchanges small typed **frames**. Seven frame types cover the full lifecycle:

Frame	Purpose
TOKEN_SYNC	Broadcast an LLM answer to all peers so they cache it
HEALTH	Periodic CPU / RAM / disk / latency snapshot (every 2 s)
SIGNAL	Alerts and topology-change announcements
CONFIG	Runtime configuration pushed to all nodes
METRIC	Token-usage statistics aggregated across the cluster
VECTOR / DELTA	Tensor and model-delta transport (extension use)

Frames travel over HTTP by default; durable transports (Kafka, NATS) are available as a commercial extension. The protocol carries no central coordinator — every node is a peer.

2.2 The five-step token pipeline

Each query is processed through an ordered pipeline. Every step is additive: a query that misses the caches still benefits from compression.

1. LOCAL CACHE In-process semantic lookup (sub-millisecond).
Hit -> return, 0 tokens.
2. CLUSTER CACHE Local copy of cluster-wide answers, kept current by
TOKEN_SYNC frames. Hit -> return, 0 tokens, no network call.
3. COMPRESSION Rank retrieved chunks by cosine similarity to the query;
keep top-K (default 3), drop the rest before prompting.
4. DEDUPLICATION Coalesce concurrent identical in-flight queries onto a
single shared future.
5. LLM CALL + SYNC One LLM call with compressed context. Cache locally,
then broadcast a TOKEN_SYNC frame to all peers.

The semantic match in steps 1–2 is what distinguishes this from exact-match caches: paraphrases resolve to the same entry above a configurable similarity threshold.

2.3 Cache-sharing as failover

Because every node already receives a steady stream of HEALTH and TOKEN_SYNC frames from its peers, *silence is itself a signal*. The cluster runs a three-tier topology:

- **GREEN** — the active node serving traffic.
- **BLUE** — a warm standby. It has been receiving GREEN's TOKEN_SYNC frames all along, so its cache is already populated. A watchdog promotes BLUE to active if GREEN's frames stop arriving past a silence threshold.
- **ORANGE** — a syncing reserve, typically on a separate network.

No quorum, no external consensus store. The standby is "warm" precisely because the cache-sharing channel doubles as a liveness channel.

2.4 Novelty

The individually familiar pieces here are semantic caching and hot-standby failover. The contribution we believe is novel is **unifying them onto one peer-to-peer frame protocol**: the cache-replication traffic is the failure detector, so a warm standby comes "for free" as a side effect of cache sharing, with no separate heartbeat subsystem and no consensus dependency. This is the design documented for provisional patent purposes.

3. Benchmark methodology

***Scope statement.** These results come from a three-node cluster on Azure Container Apps, not a large production deployment. They demonstrate that the mechanisms work and quantify their behavior on a controlled workload — not to model the savings any specific production system will see. Your numbers will depend on your query-repetition rate, cluster size, and document sizes.*

Setup. Three PrismNode containers on **Azure Container Apps (westus2)**:

- node-green (active) and node-blue (warm standby) in Container App Environment A — one virtual network.
- node-orange (reserve) in Container App Environment B — a separate virtual network.
- An external benchmark runner issued queries over HTTPS from a fourth host, cross-network to all three nodes.

Network. GREEN↔BLUE communicate within one VNet; ORANGE communicates cross-VNet. Both environments are in the **same Azure region (westus2)**, so the cross-VNet latency reported is same-region; cross-region latency is not measured here. Token-savings numbers are a function of program logic, not network distance.

Workload. Five unique knowledge-base questions, each with ten candidate context chunks available for compression. GREEN answered the five unique queries; BLUE and ORANGE were then asked a subset of those same queries.

Reproducibility. The full harness is open source: `benchmark/cluster/run_cluster_benchmark.py`; the deploy script is `deploy/azure_cluster_run.sh`; raw output is committed to `benchmark/cluster/cluster_benchmark_results_azure.json`.

4. Results

4.1 Token savings across the cluster

Node	Role	Network	Tokens billed	Tokens saved	Mechanism	Per-node savings
node-green	active	same-pod	328	130	context compression	28.4%
node-blue	warm standby	same-pod	0	61	cluster cache	100%
node-orange	reserve	cross-network	0	75	cluster cache	100%

How to read this honestly. GREEN paid full price for context but compressed it, cutting its own usage by 28.4%. BLUE and ORANGE billed **zero tokens** because the queries they received had already been answered by GREEN and broadcast to them via TOKEN_SYNC before they were asked.

The arithmetic mean of the three per-node figures is **76.1%**, and we use that headline number elsewhere — but it is only meaningful under this paper's workload, where warm nodes receive already-answered queries. A more conservative reading: **the savings on any given node equal the fraction of its queries that some other node has already answered, plus ~28–64% context compression on the queries that remain genuinely novel.** In a cluster with little query repetition, savings trend toward the compression-only figure; in a cluster with high repetition (FAQs, support, search), they trend toward the cache figure. Both bounds are reported here rather than just the favorable one.

4.2 Context compression

Measured independently across the five queries:

Query	T Tokens used	T Tokens saved	Compression
What is PrismLib?	71	126	64.0%
How does CHORUS Fabric work?	85	118	58.1%
Explain context compression.	75	128	63.1%
What is Blue/Green/Orange failover?	84	116	58.0%
How does token deduplication work?	81	116	58.9%
Average			60.4%

Compression is pure in-process cosine-similarity ranking over chunk embeddings — no second LLM call, no separate compression model. This is the savings floor that applies even to genuinely novel queries.

4.3 CHORUS frame latency (Azure Container Apps, westus2)

Node	Network type	Avg latency	Min	Max
node-green	same VNet	19.4 ms	15.0 ms	29.0 ms
node-blue	same VNet	19.9 ms	15.2 ms	22.6 ms
node-orange	cross-VNet	21.6 ms	16.0 ms	26.3 ms

These are real cloud measurements. Notably, cross-VNet frames (ORANGE) average only ~2 ms more than same-VNet frames — because both environments sit in the same region. Across regions this gap would widen; we do not measure that here. Frames are small and cheap either way.

4.4 Health-alert propagation (Azure)

Event	Source -> Dest	Network	Propagation
cpu_high (92%)	GREEN -> BLUE	same VNet	633 ms
cpu_high (92%)	GREEN -> ORANGE	cross-VNet	674 ms

A threshold breach fired a SIGNAL frame that reached both peers in well under a second — without a scrape interval to wait on. The comparison point is a metrics-scrape system (typically 30–60 s to alert), not a purpose-built message bus.

4.5 Failover (Azure)

Metric	Value
GREEN silence threshold	3 000 ms
Failover detected (silence crossed)	3 960 ms
Promotion to active (once detected)	97 ms
Failovers triggered	1
Human intervention	none

GREEN's heartbeat was paused. BLUE's watchdog observed the silence, crossed the 3 s threshold at ~3.96 s, and completed promotion to active 97 ms later — announcing the new topology via a SIGNAL frame. The ~4 s figure is dominated by the deliberately conservative silence threshold, not the promotion itself; tightening the threshold to 1 s would bring total failover under ~1.2 s at the cost of more sensitivity to transient network blips.

4.6 Real-world Azure deployment — cache & driver layers

The two single-node layers that PrismLib Micro builds on were deployed to **Azure Container Apps (westus2)** and load-tested with a concurrent client. Where §4.1–4.5 exercise the mesh with a functional 3-node run, this section puts the cache and driver under **sustained concurrent traffic**.

PrismCache — "mixed" scenario, 50 concurrent users, 300 s, 6,973 queries:

Metric	Value
Cache hit rate	95.9%
Hits / misses	6,687 / 286
Tokens saved (run)	1,673,216
Projected monthly cost saved	~\$723
Avg hit latency	291 ms
Avg miss latency	415 ms

A lighter run (20 users, 60 s) measured a 91.0–92.0% hit rate, confirming the 91–96% range holds across load levels.

PrismDriver — 30 concurrent users × 60 s per phase, 11,000-row index, Azure Container Apps westus2:

Metric	Baseline (DB over network)	PrismDriver (local index)
Avg read latency	142.83 ms	2.02 ms
Speedup	—	70.7×
Latency reduction	—	98.6%

These two results validate the cache and driver layers on production-grade cloud infrastructure under concurrent load, complementing the cluster-mesh results in §4.1–4.5 (also on Azure, but a functional 3-node run rather than a sustained load test).

5. Threats to validity

We list these so the reader does not have to reverse-engineer them.

1. **Small cluster.** Three nodes. Behavior at 50+ nodes — frame fan-out cost, cache memory growth, broadcast storms — is not measured here.
2. **Single region.** The cluster ran on Azure Container Apps across two VNets, but both in westus2. Cross-region latency, packet loss, and network partition behavior are untested. The ~22 ms cross-VNet latency in §4.3 is a same-region figure and would be higher across regions.

- 3. **Favorable cache workload.** The 100% warm-node savings depend on warm nodes receiving already-answered queries. The headline 76.1% mean is the most favorable framing of the data; §4.1 gives the conservative bounds.
- 4. **Small absolute token counts.** Queries were short (60–85 tokens). Ratios should hold for larger contexts but were not measured at scale.
- 5. **No consensus guarantees.** Leaderless promotion trades Raft's correctness guarantees for simplicity. A network partition could in principle promote two actives ("split brain"); the current design targets workloads where a brief duplicate-active window is acceptable, not strongly-consistent stores.
- 6. **Single run.** Numbers are from one execution, not averaged over many with variance reported.

None of these invalidate the mechanisms; they bound the claims.

6. Related approaches

Concern	Conventional tool	Trade-off vs PrismLib Micro
Cross-node cache	GPTCache + Redis	External store; exact-or-embedding match per process, not auto-shared
Prompt caching	Provider prefix cache	Prefix-only, per-API-key, not cross-node
Health alerting	Prometheus / Datadog	30–120 s scrape latency; extra stack or per-host cost
Failover	Raft (etcd/consul)	150–500 ms election and strong consensus — stronger than ours — but an extra cluster to run
Context compression	LLMLingua	Higher compression, but needs a GPU-hosted model

We do not claim to beat Raft on failover correctness or LLMLingua on compression ratio. The argument is one of *consolidation*: acceptable numbers on each axis with zero added infrastructure, by reusing one protocol.

7. Conclusion

Redundant LLM work across a cluster is a structural cost, not a tuning problem. PrismLib Micro removes it by sharing answers over a peer-to-peer frame protocol and, by reusing that same protocol's traffic as a liveness signal, provides warm-standby failover as a side effect rather than a separate subsystem. The benchmark — run on Azure Container Apps across two VNets — shows the mechanisms working: zero-token serving on warm nodes, 58–64% context compression on novel queries, ~22 ms cross-VNet frames, sub-700 ms alerting, and ~4 second leaderless failover (97 ms of which is the actual promotion) — with the explicit caveat that these come from a small, same-region cluster and bound, rather than predict, production behavior at scale.

The core library is open source under Apache 2.0. The benchmark is reproducible from the repository.

Appendix A — Reproducing the benchmark

```
# Deploy 3 nodes to Azure Container Apps (2 VNets) and run the benchmark:
bash deploy/azure_cluster_run.sh
# raw output: benchmark/cluster/cluster_benchmark_results_azure.json

# Or run locally against your own node URLs:
python benchmark/cluster/run_cluster_benchmark.py \
  --green <url> --blue <url> --orange <url>
```

Appendix B — Artifacts

- `deploy/azure_cluster_run.sh` — Azure Container Apps deploy + benchmark
- `benchmark/cluster/run_cluster_benchmark.py` — five-phase harness
- `benchmark/cluster/cluster_benchmark_results_azure.json` — raw Azure results
- `prism/cluster/cache.py` — ClusterCache, ContextCompressor
- `prism/cluster/node.py` — PrismNode, failover watchdog
- `prism/lib/fabric.py` — CHORUS frame protocol

Appendix C — Contact

- Email: insightits.info@gmail.com
- Repository: <https://github.com/insightitsGit/prismlib>
- PyPI: <https://pypi.org/project/prismlib>
- License: Apache 2.0